Module V

Part II - Solidity

The Solidity Language

Solidity is a language for programming contracts in Ethereum.

It is a statically typed language, which means that variable type checking in Solidity is carried out at compile time.

It has features like inheritance, libraries, and the ability to define composite data types.

Solidity is also called a contract-oriented language.

In Solidity, contracts are equivalent to the concept of classes in other object-oriented programming languages.

Layout of a Solidity Source File

A Solidity source file can contain any number of contracts/library definitions in a single file.

The Solidity contract/library file extension is *.sol*, for example, *SampleContract.sol*.

The basic layout of the Solidity source file contains following in order:

- ★ The Solidity version to use
- ★ Experimental pragma if any
- ★ Import statements to import other contracts/libraries
- ★ Contract definition.

Solidity version with pragma

Each Solidity contract/library source file should have the version of Solidity compiler specified.

The version of the compiler should be specified first thing in the source file.

```
pragma solidity ^0.4.4;
```

If a source file is of this version, it means the file will compile with 0.4.4 and higher versions in the 0.4.x branch, up to version 0.5.0 where 0.5.0 is not included.

It is recommended that the Solidity version should be locked and specified without the ^ (caret) sign, as follows:

```
pragma solidity 0.4.4;
```

This ensures that the specific compiler version will always be used.

Importing other source files

Solidity supports an import statement to import other source files.

```
import "path/to/file/filename.sol";
```

In the preceding example, the absolute path of the file should be specified.

Comments

Comments can be added in the Solidity source code file in a manner similar to C-language.

Multiple line comments are enclosed in /* and */, whereas single line comments start with //.

```
pragma solidity ^0.4.0; //specify the compiler version
/*
This is a simple value checker contract that checks the value
provided and returns boolean value based on the condition
expression evaluation.
*/
import "dev.oraclize.it/api.sol";
contract valuechecker {
    uint price=10;
    //This is price variable declare and initialized with value 10
    event valueEvent(bool returnValue);
    function Matcher (uint8 x) returns (bool)
    {
        if ( x >= price)
        {
            valueEvent(true);
            return true;
        }
    }
}
```

A sample solidity program

Structure of a contract

Just like in an object-oriented language such as Java, where we have classes, we have contracts in Solidity languages.

Each Solidity contract can contain declarations of one or many state variables, constants, a constructor function, enumerations types, function modifiers, events, functions, and struct types.

Furthermore, a contract can inherit from other contracts as well.

We can also define special kinds of contracts called libraries and interfaces.

Libraries are deployed only once along with contract deployment.

Interfaces cannot have function definitions, variables, struct types, or enums.

Normally, the structure of these constructs is maintained in the following order.,

1. Global state variables of contract
2. Enum types
3. Struct types
4. Function modifier definitions
5. Events definitions
6. Functions

1. Declaring state variables

The state variables defined in contract are values that are permanently stored in contract storage over the Ethereum blockchain.

The variables defined outside of the method/function body are accessible to the whole contract.

Defining a `balanceAmount` variable of the uint type is done as shown in the following code:

```
contract VariableStorage {
    uint balanceAmount; //uint256 storage variable
    //...
}
```

2. Custom types for constants with enum

Enums can be used to create a custom data type with a fixed set of constant values.

Enums will have integer values assigned, starting from 0 and ascending to the number of states defined in enum.

```
contract Loan {
   //Enum for LoanStatus
    enum LoanStatus { Pending,
                      Created,
                      Funded,
                      Finished,
                      Defaulted
                    }
    LoanStatus status; //Enum variable declaration
    function setValue() public {
                                status = LoanStatus.Created;
                               }
 }
```

3. Custom data types with struct

Structs are custom-defined data types that can group multiple variables in a single struct variable to form a new type.

By default, all struct values are initialized using their default values.

For example, uint values will be initialized with 0; bool values will be initialized with false; and address values will be initialized with 0x0.

```
contract LoanStruct {
    //Definition of struct
    struct LoanData {
        address borrower;
        address lender;
        uint256 loanAmount;
    }
}
```

4. Function modifiers

The modifiers are used to check the pre- and post-conditions before calling a method.

Argument validation and correctness are checked with the modifiers.

The function modifiers are also used to maintain the access and role-based control on the contract.

In the following example code, *onlyOwner* is an access modifier function used in the withdraw function.

The definition says that the withdraw function can only be allowed to call from the owner address:

```
contract AccountContract {
    address public owner;
    modifier onlyOwner() {
        require(msg.sender == owner, "not owner");
        _;  //Rest of the function body execution
    }
    function withdraw() public onlyOwner {
        msg.sender.transfer(address(this).balance);
}
```

Function modifier definition

Modifier called along with 'withdraw()'

Function modifier Example

5. Events for logging

Solidity contracts do not have a way to generate logs when they are executed.

Events are used to log values of the variables.

When events are emitted, its data is stored in the transaction's log.

Events also provide a way to maintain action history.

6. Writing function definitions

The Solidity functions are executable units of code within a contract.

These are the only access points for the users and the different entities that will interact with a contract.

There can be many functions within a contract.

A deposit function is defined as shown here.

Your contract can receive ether along with a method call using the payable keyword, as shown in the following code:

modifier

```
contract AcceptEther {
    function deposit() public payable {
                        //function example to accept ETH
                        //...
    }
}
```

Solidity data types

The Solidity language is statically typed.

When writing a contract or library definition, all of the state variables and local variables must have the data type specified along with its declaration.

We can create your own custom type by using the data types supported by Solidity; for example, using the struct variable.

In Solidity, there is no concept of null values.

Instead of null, the default value of each data type is defined.

Solidity type	Default value
uint8 - uint256	0 (zero)
int8 - int256	0 (zero)
bool	false
string	An empty string
byte	In integer form, 0; in hex form, 0x00
bytes1 - bytes32	In integer form, 0; in hex form, 0x<NumberOfZeros> (*NumberOfZeros = NumberOfBytes * 2;* for example, for bytes2, the default value is 0x0000)
bytes	Empty bytes prefixed with 0x
address	0x0000000000000000000000000000000000000000

Solidity Data types and Default Values

Solidity's data types are further divided into the following:

Value types: The data types whose values are copied when passed as method arguments. These are int, uint, bool, and address.

Reference types: The data types whose references are copied when passed as method arguments. These are array and struct.

Some int and uint value types in Solidity support various sizes to reduce gas consumption and storage space.

You should choose the appropriate value type based on your data requirements.

The *bool* and *address* value types do not have different sizes, as their size is fixed.

The size of the value types is as follows:

Integer values

Value type	Different sizes	When size is undefined
`int`	`int8`, `int16`, `int24` .... `int256` (a step of 8)	`int` uses `int256`
`uint`	`uint8`, `uint16`, `uint24` .... `uint256` (a step of 8)	`uint` uses `uint256`

The different sizes of value types supported

Boolean value type

Solidity supports the bool data type that can store values such as true or false.

```
bool isInitialized = false;
```

Address value type

The address data type represents the public address of an external account or a smart contract.

An address data type stores 20 bytes value that represents an Ethereum address.

An Ethereum address is always represented in hexadecimal and prefixed with 0x.

The address looks like: 0xFf899Af34214B0D777bcD3c230EF637B763F0B01.

```
address owner = 0xFf899Af34214B0D777bcD3c230EF637B763F0B01;
//Fixed value without quotes

address emptyAddress = address(0);  //Sets the address to
0x0

address current = address(this); //Assigns current
contract's address
```

Reading a contract's ether balance

The address type has only one member property, that is, .balance.

The syntax is as follows:

<address>.balance

.balance is a property; it is not a member function.

This returns the current ether balance of an address as uint256 and is represented in wei.

```
//This will return Ether balance of function caller.

uint256 balanceInWei = msg.sender.balance;
```

Sending ether using ‘transfer’

The syntax for using transfer is as follows:

```
<address>.transfer(uint256 amount)
```

.transfer is used to transfer ether units in wei to another external account or contract account.

In the call, address is the public address of external account or contract account, and the amount parameter is always specified in wei.

Sending ether using send

```
<address>.send(uint256 amount) returns (bool)
```

The .send() function is used to send ether units in wei to another external account or contract account.

This is same as the address.transfer function call.

The only difference is that this call returns a bool result to indicate that the .send operation succeeded or not.

Solidity control structures

Used for conditional and repeated code execution.

Conditions should be in parentheses (...) and code blocks should be in curly brackets {...}.

1. The following are conditional control structures:

if...else block: For conditional execution of the logic.

We can also have nested if...else structures.

? : operator: A ternary operator for conditional checking in a single statement.

NB: Give examples from C

2. The following are iteration control structures (loops):

while loop: For creating while loops in Solidity

for loop: Used to create for loops

do...while loop: Also supports do...while loop constructs

NB: Give examples from C

3. The following are jump control structures:

break keyword: To exit from the loop that is currently being executed on a certain condition

continue keyword: To continue the loop for the next iteration upon a certain condition

return keyword: To return one or more value from a function

Events

Events in Solidity can be used to log certain events in EVM logs.

These are quite useful when external interfaces are required to be notified of any change or event in the contract.

These logs are stored on the blockchain in transaction logs.

Logs cannot be accessed from the contracts but are used as a mechanism to notify change of state or the occurrence of an event (meeting a condition) in the contract.

```
pragma solidity ^0.4.0;
contract valueChecker
{
uint8 price=10;
event valueEvent(bool returnValue);
function Matcher(uint8 x) public returns (bool)
    {
        if (x>=price)
            {
                valueEvent(true);
                return true;
            }
    }
}
```

The valueEvent event will return true if the x parameter passed to function Matcher is equal to or greater than 10

Inheritance

Inheritance is supported in Solidity.

The is keyword is used to derive a contract from another contract.

In the following example, *valueChecker2* is derived from the *valueChecker* contract.

The derived contract has access to all non-private members of the parent contract:

```
pragma solidity ^0.4.0;
contract valueChecker
{
    uint8 price = 20;
    event valueEvent(bool returnValue);
    function Matcher(uint8 x) public returns (bool)
    {
        if (x>=price)
        {
            valueEvent(true);
            return true;
        }
    }

}
contract valueChecker2 is valueChecker
{
    function Matcher2() public view returns (uint)
    {
        return price+10;
    }
}
```

Keyword for inheritance

Example for Inheritance

Libraries

Libraries are deployed only once at a specific address and their code is called via CALLCODE or DELEGATECALL opcode of the EVM.

The key idea behind libraries is code reusability.

They are similar to contracts and act as base contracts to the calling contracts.

A library can be declared as shown in the following example:

```
library Addition
{
    function Add(uint x,uint y) returns (uint z)
    {
        return x + y;
    }
}
```

This library can then be called in the contract, as shown here.

First, it needs to be imported and then it can be used anywhere in the code.

```
import "Addition.sol"
function Addtwovalues() returns(uint)
{
    return Addition.Add(100,100);
}
```

Imports library example

Limitations of Library

- They cannot have state variables
- Cannot inherit or be inherited.
- The cannot receive Ether

Functions

Functions in Solidity are modules of code that are associated with a contract.

Functions are declared with a name, optional parameters, access modifier, optional constant keyword, and optional return type.

Keyword to declare function

Function name

Parameter (optional)

```
function orderMatcher (uint x)
private constant returns(bool return value)
```

Access specifier

keyword

Return type (optional)

’constant’ - used to specify that this function does not change anything in the contract but is used only to retrieve values from the contract

access modifier or specifier controls access to the function from external contracts.

- **How to define a function**: The syntax of defining a function is shown as follows:

```
      function <name of the function>(<parameters>) <visibility specifier>
returns
      (<return data type> <name of the variable>)
      {
          <function body>
      }
```

Function signature: Functions in Solidity are identified by its signature, which is the first four bytes of the Keccak-256 hash of its full signature string.

Input parameters of a function

```
contract myContract
{
    function checkValues(uint x, uint y)
    {
    }
}
```

Output parameters of a function

```
contract myContract
{
    function getValue() returns (uint z)
    {
        z=x+y;
    }
}
```

a function can return up to 14 values of different data types

Internal function calls: Functions calls to functions in the current contract.

External function calls: Function calls from a contract to another contract.

Fallback functions: This is an unnamed function in a contract with no arguments and return data.

This function executes every time Ether is received.

It is required to be implemented within a contract if the contract is intended to receive Ether.

Modifier functions: They are used to check some conditions or verification before executing the function.

Constructor function: This is an optional function that has the same name as the contract and is executed once a contract is created.

There is only one constructor allowed in a contract.

Function visibility specifiers (access modifiers):

External: These functions are accessible from other contracts.

Public: By default, functions are public. They can be called either internally or using messages.

Internal: Internal functions are visible to other derived contracts from the parent contract.

Private: Private functions are only visible to the same contract they are declared in.

Error handling in Solidity

In Solidity code, when certain conditions are met, you can throw an exception and fail the transactions.

You can check for method input argument's correctness and revert the transaction if arguments are not in order.

When a transaction is reverted or failed, all of its state changes and variable modifications are reverted.

Also, the gas consumption it has consumed till the revert of the transaction will be consumed and sent to miner, and the rest of the remaining gas will be refunded back to the transaction initiator.

The error handling methods are as follows:

assert(bool condition): This invalidates the transaction if the condition is not met or false

require(bool condition): This reverts the transaction if the Boolean condition is not met.

require(bool condition, string message): Same as the previous bullet point, but provides a message.

revert(): This aborts the execution of a transaction and reverts all state changes.

revert(string reason): This aborts the execution of a transaction and reverts all state changes. Also shows error message.

Case Studies:

Voting: https://docs.soliditylang.org/en/v0.8.17/solidity-by-example.html#voting

Simple open auction:
https://docs.soliditylang.org/en/v0.8.17/solidity-by-example.html#simple-open-auction

You can find some other use cases too from the above URL.